

BinX Tech

Backend .NET Internship

Sprint 3 — Day 5

Sprint Review & Retrospective

Benchmark Demo & Sprint 3 Close-Out

Project

Cardiac Patient Monitoring System

Focus

Queries • Redis • Indexing

13 → 1

SQL Queries

783 → 34

Cache Hit Timing

0.259 → 0.102

Index Timing

Section 1

Sprint 3 Review

Sprint Goal	Improve application and database performance by reducing unnecessary EF Core database round trips, introducing Redis caching, and validating PostgreSQL indexing with real execution-plan evidence.
--------------------	---

Day 5 Purpose	Consolidate actual evidence from Days 1–4, demonstrate measurable before/after improvements, review Sprint 3 acceptance criteria, identify remaining work, and define one concrete Sprint 4 action.
----------------------	---

Sprint Objectives

- Identify and remove the N+1 query pattern in the appointment summary endpoint.
- Compare EF Core Include and Projection using generated SQL and measurements.
- Introduce Redis Cache-Aside caching for the paginated Patients endpoint.
- Verify cache hits and invalidation after Patient Create, Update and Delete operations.
- Add a PostgreSQL composite index for the Patients filtering and ordering pattern.
- Use EXPLAIN (ANALYZE, BUFFERS) to compare database execution plans.
- Document concrete before/after performance evidence.

Sprint Review Checklist

Appointment Summary Baseline	COMPLETED
GET /api/appointments/summary was implemented and verified through Postman with a successful 200 OK response.	
N+1 Query Identification	COMPLETED
Six appointments produced 13 SQL queries: one Appointments query, six Patient queries and six Doctor queries.	
EF Core Query Optimization	COMPLETED
Include and Projection both reduced appointment-summary database access to one SQL query.	
Redis Caching	COMPLETED
Cache-Aside was implemented using IDistributedCache with StackExchange.Redis and a 10-minute expiration.	
Cache Invalidation	COMPLETED
Patient Create, Update and Delete increment the Redis cache version.	
PostgreSQL Indexing	COMPLETED

Gender + DateOfBirth composite index was added and validated through EXPLAIN (ANALYZE, BUFFERS).

Performance Evidence

COMPLETED

Query counts, cache hit/miss timings and execution-plan evidence were recorded.

Sprint 3 Pull Request / Review Flow

PENDING

The formal GitHub PR/review/merge close-out step remains to be completed.

Section 2 — Benchmark Demo

Before / After Performance Evidence

2.1 Appointment Summary — N+1 Query Problem

The original implementation retrieved appointments and then loaded related Patient and Doctor records separately inside the loop. With six appointments, this produced 13 SQL queries.

Implementation	SQL Queries	SQL Time	HTTP Time	Result
Original N+1	13	Multiple queries	183 ms	Repeated related-data queries
Include	1	11 ms	229 ms	N+1 eliminated
Projection	1	7 ms	199 ms	N+1 eliminated; DTO fields only

Key Result

Database round trips were reduced from **13 SQL queries to 1 SQL query**. Projection was the leaner option for the summary endpoint because it selects only the fields required by the DTO.

2.2 Redis Cache — Hit / Miss Benchmark

Scenario	Observed Timing	Database Evidence	Cache State
Initial GET /api/patients	783 ms	Patients SELECT executed	MISS
Same GET again	34 ms	No new Patients SELECT	HIT
GET after update	16 ms	New DB query executed	MISS
Same GET again	3 ms	No new SELECT	HIT

Comparison	Reduction
783 ms → 34 ms	≈95.7%
16 ms → 3 ms	81.25%

Benchmark Note

The captured timings are local measurements and can vary with database state, Redis state and machine load. The stronger structural evidence is that a cache hit produced no new Patients SELECT.

Section 3 — Redis Caching

Caching Strategy & Invalidation

Caching Target	Paginated Patients list endpoint — a read-heavy endpoint where identical query parameters can reuse cached results.
----------------	---

Component	Implementation
Abstraction	ASP.NET Core IDistributedCache
Provider	StackExchange.Redis
Pattern	Cache-Aside
Expiration	10 minutes
Cache Key	Version + page + pageSize + normalized search + normalized gender + normalized sort
Cache Hit	Return cached response without issuing a new Patients SELECT
Cache Miss	Query PostgreSQL, store result in Redis, then return response
Invalidation	Increment Patients cache version after successful Create, Update and Delete

Observed Invalidation Evidence

Operation	Observed Evidence
PUT /api/patients/15	UPDATE executed; Redis cache version incremented to 3; next GET queried the database again.
POST /api/patients	INSERT executed; Redis cache version incremented to 4; next GET queried the database again.
DELETE /api/patients/17	DELETE executed; Redis cache version incremented to 5; invalidation confirmed.

Why Versioning?
The version-based key strategy invalidates previous cache entries logically by changing the version component of the key. New requests therefore do not reuse entries created under the previous version.

Section 4 — Database Performance

PostgreSQL Indexing & Performance Profiling

Dataset 1,000 synthetic Patient records were used to profile the targeted query.

Composite Index

```
CREATE INDEX "IX_Patients_Gender_DateOfBirth" ON public."Patients" USING btree ("Gender", "DateOfBirth");
```

Profiled Query

```
SELECT * FROM "Patients" WHERE "Gender" = 'Male' ORDER BY "DateOfBirth" LIMIT 10 OFFSET 0;
```

Execution Plan Comparison

State	Execution Plan Evidence	Execution Time
Before index usage	Seq Scan on Patients; Rows Removed by Filter: 507	0.259 ms
After index usage	Index Scan using IX_Patients_Gender_DateOfBirth; Index Cond: (Gender = 'Male'); Buffers: shared hit=11	0.102 ms

Before	After	Observed Reduction
0.259 ms	0.102 ms	≈60.6%

Primary Evidence

The most important result is the execution-plan change from **Seq Scan** to **Index Scan**. The measured timing is a local benchmark and may vary between environments.

Section 5 — Sprint 3 Summary

Consolidated Performance Results

Performance Area	Before	After	Impact
Appointment Summary / N+1	13 SQL queries	1 SQL query	≈92.3% fewer queries
Appointment Summary / Projection	13 SQL queries	1 query; 7 ms SQL	Lean DTO-oriented query
Redis Cache	783 ms miss	34 ms hit	≈95.7% lower observed time
Redis Repeated Hit	16 ms miss	3 ms hit	81.25% lower observed time
Patients Index	Seq Scan; 0.259 ms	Index Scan; 0.102 ms	≈60.6% lower captured time

Evidence Interpretation

- The N+1 optimization is demonstrated primarily by the reduction in database round trips: 13 queries to 1.
- Redis caching is demonstrated by the absence of a new Patients SELECT on repeated cache-hit requests.
- Redis invalidation is demonstrated by a new database query after Patient write operations.
- PostgreSQL indexing is demonstrated by the execution-plan change from sequential scan to index scan.
- HTTP and local SQL timings are supporting measurements and can vary with local environment conditions.

Section 6 — Iteration Backlog

Sprint 4 Backlog Handoff

Only remaining work supported by the current Sprint 3 evidence is carried forward. No additional performance issue is invented for the backlog.

Backlog Item	Reason / Target	Priority
Complete Sprint 3 PR / review / merge flow	Close the formal GitHub review process required for sprint completion.	High
Automated query-count regression coverage	Protect the appointment summary endpoint from silently returning to the N+1 pattern.	High

Sprint 4 Concrete Action

Action

Add an automated performance-regression test that verifies the appointment summary path does not reintroduce the N+1 pattern and keeps the related-data query count at the optimized level.

Definition of the Action

- Convert the manually observed query-count check into automated regression protection.
- Keep the optimized appointment summary behavior at one related-data query.
- Use the test result as evidence during future Sprint Reviews.

Section 7 — Retrospective

Sprint 3 Retrospective

What Went Well

- Performance work was based on actual application behavior and captured evidence.
- EF Core SQL logging made the N+1 query pattern directly visible.
- Optimization was validated through SQL query count rather than relying only on HTTP timing.
- Redis caching was implemented with explicit invalidation for Create, Update and Delete operations.
- Cache hits and misses were verified through response timing and database-query behavior.
- PostgreSQL indexing was validated using EXPLAIN (ANALYZE, BUFFERS).
- The sprint produced measurable improvements across query count, caching and database execution planning.

What Needs Improvement

- Complete the formal GitHub PR/review/merge process before considering the sprint fully closed.
- Convert important manual performance checks into automated regression tests where practical.
- Keep benchmark interpretation focused on structural evidence such as query counts and execution plans, not only raw timing.

One Concrete Action for Sprint 4

Sprint 4 Action

Automate the appointment-summary query-count check so an N+1 regression is detected automatically instead of being discovered manually during performance profiling.

Retrospective Outcome

Sprint 3 established a practical performance workflow: observe the real behavior, identify the bottleneck, apply an optimization, measure the result, and record the evidence. Sprint 4 should strengthen this workflow by turning the most important performance assumption into an automated regression check.

Section 8 — Sprint Close-Out

Sprint 3 Close-Out Checklist

Close-Out Item	Status
Before/after benchmark evidence documented	COMPLETED
N+1 optimization demonstrated	COMPLETED
Redis cache hit/miss demonstrated	COMPLETED
Cache invalidation verified for Create / Update / Delete	COMPLETED
PostgreSQL index and execution-plan evidence documented	COMPLETED
Sprint Review recorded	COMPLETED
Sprint 4 backlog item recorded	COMPLETED
Sprint Retrospective recorded	COMPLETED
One concrete Sprint 4 action defined	COMPLETED
GitHub PR opened / reviewed / merged	PENDING

Final Sprint Summary

Sprint 3 delivered measurable performance improvements across three complementary layers: EF Core query optimization, Redis caching and PostgreSQL indexing. The appointment summary endpoint moved from a classic N+1 pattern of 13 SQL queries to a single query. The Patients list endpoint demonstrated significant cache-hit response-time reductions and verified cache invalidation after writes. PostgreSQL profiling showed a change from sequential scanning to index scanning for the targeted filter-and-order pattern.

The technical objectives of Sprint 3 are demonstrated with real evidence. The remaining process-level close-out item is the formal GitHub PR/review/merge flow. Sprint 4 should focus on protecting the achieved optimization through automated regression coverage.

Sprint 3 Result

Query efficiency improved • Redis caching verified • Database indexing validated • Performance evidence documented • Sprint 4 action defined

Evidence Sources Used

- Week 8 Day 1 — Appointment Summary baseline and N+1 identification.
- Week 8 Day 2 — EF Core Include / Projection optimization and measurements.
- Week 8 Day 3 — Redis caching, cache hit/miss timing and invalidation evidence.
- Week 8 Day 4 — PostgreSQL indexing and EXPLAIN (ANALYZE, BUFFERS) profiling.
- Week 8 Day 5 — Sprint Review, benchmark demonstration, backlog handoff and retrospective requirements.